

Chapter 4: Basic Control Flow

By Irina Galata

When writing a computer program, you need to be able to tell the computer what to do in different scenarios. For example, a calculator app would need to do one thing if the user tapped the addition button and another thing if the user tapped the subtraction button.

In computer-programming terms, this concept is known as **control flow**. It is so named because the flow of the program is controlled by various methods. In this chapter, you'll learn how to make decisions and repeat tasks in your programs by using syntax to control the flow. You'll also learn about **Booleans**, which represent true and false values, and how you can use these to compare data.

Open up a project in IntelliJ to follow along!

Comparison operators

You've seen a few types now, such as `Int`, `Double` and `String`. Here you'll learn about another type, one that will let you compare values through the **comparison operators**.

When you perform a comparison, such as looking for the greater of two numbers, the answer is either *true* or *false*. Kotlin has a data type just for this! It's called a `Boolean`, after a rather clever man named George Boole who invented an entire field of mathematics around the concept of true and false.

This is how you use a `Boolean` in Kotlin:

```
val yes: Boolean = true
val no: Boolean = false
```

And because of Kotlin's type inference, you can leave off the type annotation:

```
val yes = true
val no = false
```

A `Boolean` can only be either true or false, denoted by the keywords `true` and `false`. In the code above, you use the keywords to set the state of each constant.

Boolean operators

Booleans are commonly used to compare values. For example, you may have two values and you want to know if they're equal: either they are (true) or they aren't (false).

In Kotlin, you do this using the **equality operator**, which is denoted by `==`. Try it out:

```
val doesOneEqualTwo = (1 == 2)
```

Kotlin infers that `doesOneEqualTwo` is a `Boolean`. Clearly, 1 does not equal 2, and therefore `doesOneEqualTwo` will be `false`.

Similarly, you can find out if two values are *not* equal using the `!=` operator:

```
val doesOneNotEqualTwo = (1 != 2)
```

This time, the comparison is true because 1 does not equal 2, so `doesOneNotEqualTwo` will be `true`.

The prefix `!` operator, also called the **not-operator**, toggles `true` to `false` and `false` to `true`. Another way to write the above is:

```
val alsoTrue = !(1 == 2)
```

Because 1 does not equal 2, `(1 == 2)` is `false`, and then `!` flips it to `true`.

Two more operators let you determine if a value is greater than (`>`) or less than (`<`) another value. You'll likely know these from mathematics:

```
val isOneGreaterThanTwo = (1 > 2)
val isOneLessThanTwo = (1 < 2)
```

And it's not rocket science to work out that `isOneGreaterThanTwo` will equal `false` and `isOneLessThanTwo` will equal `true`.

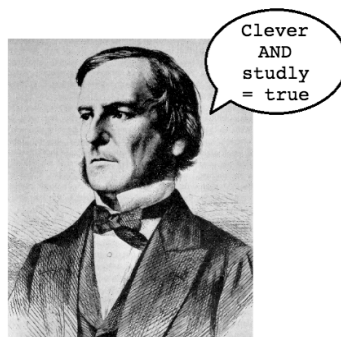
There's also an operator that lets you test if a value is less than *or* equal to another value: `<=`. It's a combination of `<` and `==`, and will therefore return `true` if the first value is either less than the second value or equal to it.

Similarly, there's an operator that lets you test if a value is greater than or equal to another — you may have guessed that it's `>=`.

Boolean logic

Each of the examples above tests just one condition. When George Boole invented the Boolean, he had much more planned for it than these humble beginnings. He invented Boolean logic, which lets you combine multiple conditions to form a result.

One way to combine conditions is by using **AND**. When you AND together two Booleans, the result is another Boolean. If both input Booleans are `true`, then the result is `true`. Otherwise, the result is `false`.



George Boole

In Kotlin, the operator for Boolean AND is `&&`, used like so:

```
val and = true && true
```

In this case, `and` will be `true`. If either of the values on the right was `false`, then `and` would be `false`.

Another way to combine conditions is by using **OR**. When you OR together two Booleans, the result is `true` if *either* of the input Booleans is `true`. Only if *both* input Booleans are `false` will the result be `false`.

In Kotlin, the operator for Boolean OR is `||`, used like so:

```
val or = true || false
```

In this case, `or` will be `true`. If both values on the right were `false`, then `or` would be `false`. If both were `true`, then `or` would still be `true`.

In Kotlin, Boolean logic is usually applied to multiple conditions. Maybe you want to determine if two conditions are true; in that case, you'd use AND. If you only care about whether one of two conditions is true, then you'd use OR.

For example, consider the following code:

```
val andTrue = 1 < 2 && 4 > 3
val andFalse = 1 < 2 && 3 > 4

val orTrue = 1 < 2 || 3 > 4
val orFalse = 1 == 2 || 3 == 4
```

Each of these tests two separate conditions, combining them with either AND or OR.

It's also possible to use Boolean logic to combine more than two comparisons. For example, you can form a complex comparison like so:

```
val andOr = (1 < 2 && 3 > 4) || 1 < 4
```

The parentheses disambiguates the expression. First Kotlin evaluates the sub-expression inside the parentheses, and then it evaluates the full expression, following these steps:

1. `(1 < 2 && 3 > 4) || 1 < 4`
2. `(true && false) || true`
3. `false || true`
4. `true`

String equality

Sometimes you want to determine if two strings are equal. For example, a children's game of naming an animal in a photo would need to determine if the player answered correctly.

In Kotlin, you can compare strings using the standard equality operator, `==`, in exactly the same way as you compare numbers. For example:

```
val guess = "dog"
val dogEqualsCat = guess == "cat"
```

Here, `dogEqualsCat` is a Boolean that in this case equals `false`, because `"dog"` does not equal `"cat"`. Simple!

Just as with numbers, you can compare not just for equality, but also to determine if one value is greater than or less than another value. For example:

```
val order = "cat" < "dog"
```

This syntax checks if one string comes before another alphabetically. In this case, `order` equals `true` because `"cat"` comes before `"dog"`.

Mini-exercises

1. Create a constant called `myAge` and set it to your age. Then, create a constant named `isTeenager` that uses Boolean logic to determine if the age denotes someone in the age range of 13 to 19.
2. Create another constant named `theirAge` and set it to the age 30. Then, create a constant named `bothTeenagers` that uses Boolean logic to determine if both you and them are teenagers.
3. Create a constant named `reader` and set it to your name as a string. Create a constant named `author` and set it to the name Richard Lucas. Create a constant named `authorIsReader` that uses string equality to determine if `reader` and `author` are equal.
4. Create a constant named `readerBeforeAuthor` which uses string comparison to determine if `reader` comes before `author`.

The if expression

The first and most common way of controlling the flow of a program is through the use of an **if expression**, which allows the program to do something only *if* a certain condition is true. For example, try typing out and running the following:

```
if (2 > 1) {  
    println("Yes, 2 is greater than 1.")  
}
```

This is a simple if expression. If the condition is true, then the expression will execute the code between the braces. If the condition is false, then the expression won't execute the code between the braces. It's as simple as that!

The term **if expression** is used here instead of **if statement**, since, unlike many other programming languages, a value is returned from the if expression. The value returned is the value of the last expression in the if block.

You are not required to use the returned value or assign it to a variable. You'll see more on returning a value below.

The else expression

You can extend an if expression to provide code to run in case the condition turns out to be false. This is known as the **else clause**. Here's an example:

```
val animal = "Fox"  
  
if (animal == "Cat" || animal == "Dog") {  
    println("Animal is a house pet.")  
} else {  
    println("Animal is not a house pet.")  
}
```

Here, if `animal` equals either "Cat" or "Dog", then the expression will run the first block of code. If `animal` does not equal either "Cat" or "Dog", then the expression will run the block inside the `else` part of the if expression, printing the following to the console:

```
Animal is not a house pet.
```

You can also use an if-else expression on one line. Let's take a look at how this can make your code more concise and readable.

If you wanted to determine the minimum and maximum of two variables, you *could* use `if` expressions like so:

```
val a = 5
val b = 10

val min: Int
if (a < b) {
    min = a
} else {
    min = b
}

val max: Int
if (a > b) {
    max = a
} else {
    max = b
}
```

By now you know how this works, but it's a lot of code. Let's take a look at how we can improve this using the fact that the `if-else` expression returns a value.

Simply remove the brackets and put it all on one line, like so:

```
val a = 5
val b = 10

val min = if (a < b) a else b
val max = if (a > b) a else b
```

In the first example, the condition is `a < b`. If this is true, the result assigned back to `min` will be the value of `a`; if it's false, the result will be the value of `b`. So `min` is set to 5. In the second example, `max` is assigned the value of `b`, which is 10.

I'm sure you'll agree that's much simpler! This is an example of **idiomatic** code, which means you are writing code in the expected way for a particular programming language. You want to use idioms whenever they make your code clearer, as it not only makes the code better, but it allows other developers familiar with the language to comprehend your code quickly.

Note: Because finding the greater or smaller of two numbers is such a common operation, the Kotlin standard library provides two functions for this purpose: `max()` and `min()`. If you were paying attention earlier in the book, then you'll recall you've already seen these.

The else-if expression

But you can go even further than that with `if` expressions. Sometimes you want to check one condition, then another. This is where **else-if** comes into play, nesting another `if` clause in the `else` clause of a previous `if` clause.

You can use `else-if` like so:

```
val hourOfDay = 12

val timeOfDay = if (hourOfDay < 6) {
    "Early morning"
} else if (hourOfDay < 12) {
    "Morning"
} else if (hourOfDay < 17) {
    "Afternoon"
} else if (hourOfDay < 20) {
    "Evening"
} else if (hourOfDay < 24) {
    "Late evening"
} else {
    "INVALID HOUR!"
}

println(timeOfDay)
```

These nested `if` clauses test multiple conditions one by one until a true condition is found. Only the code associated with that first true condition is executed, regardless of whether subsequent `else-if` conditions are true. In other words, the order of your conditions matters!

You can add an `else` clause at the end to handle the case where none of the conditions are true. This `else` clause is optional if you don't need it; in this example you *do* need it, to ensure that `timeOfDay` has a valid value by the time you print it out.

In this example, the `if` expression takes a number representing an hour of the day and converts it to a string representing the part of the day to which the hour belongs. Working with a 24-hour clock, the conditions are checked in order, one at a time:

- The first check is to see if the hour is less than 6. If so, that means it's early morning.
- If the hour is not less than 6, the expression continues to the first `else-if`, where it checks the hour to see if it's less than 12.

- Then in turn, as conditions prove false, the expression checks the hour to see if it's less than 17, then less than 20, then less than 24.
- Finally, if the hour is out of range, the expression returns that the value is invalid.

In the code above, the `hourOfDay` constant is 12. Therefore, the code will print the following:

```
Afternoon
```

Notice that even though both the `hourOfDay < 20` and `hourOfDay < 24` conditions are also true, the expression only executes and returns the first block whose condition is true; in this case, the block with the `hourOfDay < 17` condition.

Short circuiting

An important fact about `if` expressions and the Boolean operators is what happens when there are multiple Boolean conditions separated by ANDs (`&&`) or ORs (`||`).

Consider the following code:

```
if (1 > 2 && name == "Matt Galloway") {  
    // ...  
}
```

The first condition of the `if` expression, `1 > 2` is false. Therefore the whole expression cannot ever be true. So Kotlin will not even bother to check the second part of the expression, namely the check of `name`.

Similarly, consider the following code:

```
if (1 < 2 || name == "Matt Galloway") {  
    // ...  
}
```

Since `1 < 2` is true, the whole expression must be true as well. Therefore once again, the check of `name` is not executed. This will come in handy later on when you start dealing with more complex data types.

Encapsulating variables

if expressions introduce a new concept, **scope**, which is a way to encapsulate variables through the use of braces.

Imagine you want to calculate the fee to charge your client. Here's the deal you've made:

You earn \$25 for every hour up to 40 hours, and \$50 for every hour thereafter.

Using Kotlin, you can calculate your fee in this way:

```
var hoursWorked = 45

var price = 0
if (hoursWorked > 40) {
    val hoursOver40 = hoursWorked - 40
    price += hoursOver40 * 50
    hoursWorked -= hoursOver40
}
price += hoursWorked * 25

println(price)
```

This code takes the number of hours and checks if it's over 40. If so, the code calculates the number of hours over 40 and multiplies that by \$50, then adds the result to the price. The code then subtracts the number of hours over 40 from the hours worked. It multiplies the remaining hours worked by \$25 and adds that to the total price.

In the example above, the result is as follows:

1250

The interesting thing here is the code inside the if expression. There is a declaration of a new constant, `hoursOver40`, to store the number of hours over 40. Clearly, you can use it inside the if statement.

But what happens if you try to use it at the end of the above code?

```
println(price)
println(hoursOver40)
```

This would result in the following error:

```
Unresolved reference: 'hoursOver40'
```

This error informs you that you're only allowed to use the `hoursOver40` constant within the scope in which it was created.

In this case, the `if` expression introduced a new scope, so when that scope is finished, you can no longer use the constant.

However, each scope can use variables and constants from its parent scope. In the example above, the scope inside of the `if` expression uses the `price` and `hoursWorked` variables, which you created in the parent scope.

Mini-exercises

1. Create a constant named `myAge` and initialize it with your age. Write an `if` expression to print out `Teenager` if your age is between 13 and 19, and `Not a teenager` if your age is not between 13 and 19.
2. Create a constant called `answer` and use a single line `if-else` expression to set it equal to the result you print out for the same cases in the above exercise. Then print out `answer`.

Loops

Loops are Kotlin's way of executing code multiple times. In this section, you'll learn about one type of loop: the `while` loop.

If you know another programming language, you'll find the concepts and maybe even the syntax to be familiar.

While loops

A **while loop** repeats a block of code while a condition is true. You create a `while` loop this way:

```
while (/* CONDITION */) {  
    /* LOOP CODE */  
}
```

The loop checks the condition for every iteration. If the condition is `true`, then the loop executes and moves on to another iteration.

If the condition is `false`, then the loop stops. Just like `if` expressions, `while` loops introduce a scope.

The simplest `while` loop takes this form:

```
while (true) {  
}
```

This is a `while` loop that never ends because the condition is always `true`. Of course, you would never write such a `while` loop, because your program would spin forever! This situation is known as an **infinite loop**, and while it might not cause your program to crash, it will very likely cause your computer to freeze.

Here's a more useful example of a `while` loop:

```
var sum = 1  
  
while (sum < 1000) {  
    sum = sum + (sum + 1)  
}
```

This code calculates a mathematical sequence, up to the point where the value is greater than 1000.

The loop executes as follows:

- **Before iteration 1:** `sum = 1`, loop condition = `true`
- **After iteration 1:** `sum = 3`, loop condition = `true`
- **After iteration 2:** `sum = 7`, loop condition = `true`
- **After iteration 3:** `sum = 15`, loop condition = `true`
- **After iteration 4:** `sum = 31`, loop condition = `true`
- **After iteration 5:** `sum = 63`, loop condition = `true`
- **After iteration 6:** `sum = 127`, loop condition = `true`
- **After iteration 7:** `sum = 255`, loop condition = `true`
- **After iteration 8:** `sum = 511`, loop condition = `true`
- **After iteration 9:** `sum = 1023`, loop condition = `false`

After the ninth iteration, the `sum` variable is 1023, and therefore the loop condition of `sum < 1000` becomes false. At this point, the loop stops.

Repeat-while loops

A variant of the `while` loop is called the **do-while loop**. It differs from the `while` loop in that the condition is evaluated *at the end* of the loop rather than at the beginning.

You construct a do-while loop like this:

```
do {  
    /* LOOP CODE */  
} while (/* CONDITION */)
```

Here's the example from the last section, but using a repeat-while loop:

```
sum = 1  
  
do {  
    sum = sum + (sum + 1)  
} while (sum < 1000)
```

In this example, the outcome is the same as before. However, that isn't always the case; you might get a different result with a different condition. Consider the following `while` loop:

```
sum = 1  
  
while (sum < 1) {  
    sum = sum + (sum + 1)  
}
```

Consider the corresponding do-while loop, which uses the same condition:

```
sum = 1  
  
do {  
    sum = sum + (sum + 1)  
} while (sum < 1)
```

In the case of the regular `while` loop, the condition `sum < 1` is false right from the start. That means the body of the loop won't be reached because the condition is checked at the beginning! The value of `sum` will equal 1 because the loop won't execute any iterations.

In the case of the `do-while` loop, `sum` will equal 3 because the loop executes once and then the condition is checked for the first time.

Breaking out of a loop

Sometimes you want to break out of a loop early. You can do this using the `break` statement, which immediately stops the execution of the loop and continues on to the code after the loop.

For example, consider the following code:

```
sum = 1

while (true) {
    sum = sum + (sum + 1)
    if (sum >= 1000) {
        break
    }
}
```

Here, the loop condition is `true`, so the loop would normally iterate forever. However, the `break` means the `while` loop will exit once the `sum` is greater than or equal to 1000.

You've seen how to write the same loop in different ways, demonstrating that in computer programming, there are often many ways to achieve the same result.

You should choose the method that's easiest to read and conveys your intent in the best way possible. This is an approach you'll internalize with enough time and practice.

Mini-exercises

1. Create a variable named `counter` and set it equal to 0. Create a `while` loop with the condition `counter < 10` which prints out `counter is X` (where `X` is replaced with `counter` value) and then increments `counter` by 1.
2. Create a variable named `counter` and set it equal to 0. Create another variable named `roll` and set it equal to 0. Create a `do-while` loop.

Inside the loop, set `roll` equal to `Random().nextInt(6)` which means to pick a random number between 0 and 5. Then increment `counter` by 1.

Note: Add the line `import java.util.*` to the top of your Kotlin file to be able to use `Random`.

Finally, print `After X rolls, roll is Y` where `X` is the value of `counter` and `Y` is the value of `roll`. Set the loop condition such that the loop finishes when the first 0 is rolled.

Challenges

Time to test your knowledge! If you need any help along the way, check out the solutions in the materials for this chapter.

1. What's wrong with the following code?

```
val firstName = "Joe"

if (firstName == "Howard") {
    val lastName = "Lucas"
} else if (firstName == "Ray") {
    val lastName = "Wenderlich"
}

val fullName = firstName + " " + lastName
```

2. In each of the following statements, what is the value of the Boolean answer constant?

```
val answer1 = true && true
val answer2 = false || false
val answer3 = (true && 1 != 2) || (4 > 3 && 100 < 1)
val answer4 = ((10 / 2) > 3) && ((10 % 2) == 0)
```

3. Suppose the squares on a chessboard are numbered left to right, top to bottom, with 0 being the top-left square and 63 being the bottom-right square. Rows are numbered top to bottom, 0 to 7. Columns are numbered left to right, 0 to 7. Given a current position on the chessboard, expressed as a row and column number, calculate the next position on the chessboard, again expressed as a row and column number. The ordering is determined by the numbering from 0 to 63. The position after 63 is again 0.

4. Given the coefficients `a`, `b` and `c`, calculate the solutions to a quadratic equation with these coefficients. Take into account the different number of solutions (0, 1 or 2). If you need a math refresher, this Wikipedia article on the quadratic equation will help: https://en.wikipedia.org/wiki/Quadratic_formula.

Note: Add `import kotlin.math.sqrt` to the top of your file to access the `sqrt` operation.

5. Given a month (represented with a string in all lowercase) and the current year (represented with an `Int`), calculate the number of days in the month. Remember that because of leap years, “february” has 29 days when the year is a multiple of 4 but not a multiple of 100. February also has 29 days when the year is a multiple of 400.
6. Given a number as a `Double` value, determine if this number is a power of 2.

Hint: You can use `log2(number)` to find the base 2 logarithm of number. `log2(number)` will return a whole number if number is a power of two. Add `import kotlin.math.log2` to the top of your file to use `log2`.

You can also solve the problem using a loop and no logarithm.

7. Print a table of the first 10 powers of 2.
8. Given a number `n`, calculate the `n`-th Fibonacci number. (Recall Fibonacci is 1, 1, 2, 3, 5, 8, 13, ... Start with 1 and 1 and add these values together to get the next value. The next value is the sum of the previous two. So the next value in this case is $8+13 = 21$.)
9. Given a number `n`, calculate the factorial of `n`. (Example: 4 factorial is equal to $1 * 2 * 3 * 4$.)
10. Given a number between 2 and 12, calculate the odds of rolling this number using two six-sided dice. Compute it by exhaustively looping through all of the combinations and counting the fraction of outcomes that give you that value. Don’t use a formula.

Key points

- You use the Boolean data type `Boolean` to represent true and false.
- The comparison operators, all of which return a Boolean, are:

```
Equal: `==`  
Not equal: `!=`  
Less than: `<`  
Greater than: `>`  
Less than or equal: `<=`  
Greater than or equal: `>=`
```

- You can use Boolean logic with `&&` and `||` to combine comparison conditions.
- You use `if` expressions to make simple decisions based on a condition, and return a value.
- You use `else` and `else-if` within an `if` expression to extend the decision-making beyond a single condition.
- You can use a single line `if-else` expression to make your code more clear and concise.
- Short circuiting ensures that only the minimal required parts of a Boolean expression are evaluated.
- Variables and constants belong to a certain scope, beyond which you cannot use them. A scope inherits visible variables and constants from its parent.
- `while` loops allow you to perform a certain task a number of times until a condition is met.
- The `break` statement lets you break out of a loop.

Where to go from here?

Apps very rarely run all the way through the same way every time; depending on what data comes in from the internet or from user input, your code will need to make decisions on which path to take. With `if` and `else`, you can have your code make decisions on what to do based on some condition.

In the next chapter, you'll see how to use more advanced control flow statements. This will involve more loops like the `while` loop you saw in this chapter, and a new construct called the `when` expression.